after dealing with bloc and start our journey with APIs and retrieving data from an outside server..... Let's complete our journey and see how WE (the user interface people , the people who bring the data and logic into life) manage to deal with the back end and see what that endless war is about

And that will happen when we understand what is APIs and what are all those words that are connected to them mean

---

APIs (Application Programming interface) :

- it's a mediator between the UI and the backend.
  what does a mediator mean in this scenario?
  the APIs are nothing but a link , a simple link that i use to request the data or the action that i want the back end to do (e.g. get data from the DB or Store data in the DB)

Types of APIs Reqests:

GET :

Retrieving Data from DB

- from it's name we use it to **get** the data from the data base
  e.g. we are coding the profile screen of the user , this screen contains the personal data of the user (name , username, email, photo)..... where do you think those photos are coming from? do think that we have all those data stored in the device ?
  don't be silly!!! we are smarter than this
  we use the a certain API link to send to the database to make it respond to me with the data that i want and when the data return to me i ill use it and display it

POST :

Putting new data in the DB

- the phrase **post** is often heard when talking about social media , when someone posts something on social media , what does that usually mean ? ..... it means that i made a

bunch of data (photo , caption, attached a song) and then posted it in the database of the application to show it to other users ..... right?
well the concept of **POST** when talking about APIs is the same i (the developer) have a bunch of data and then i send it to the backend with POST request to put it in the DB

PATCH :

Updating already existing data in the DB

- **PATCH** is just like **POST** it takes data from me and put it in the DB, but here's the twist , **PATCH** is used in updating already existing data

PUT :

A mix of **POST** & **PATCH** :

- **PUT** is a special one , it is a mix of what both POST and PATCH usually do
  ..the whole idea is that i give the back end some data with the **PUT** request.... that mean that if the data is related to an entity that already exists... it updates it but if no such entity exists.... a new entity ill be created and the data will be put in it

DELETE :

Destroy a Resource (Remove data from DB)

API Components :

Base URL :

The Main Address of the Server

- the **Base URL** is basically the main root address of the server that we are communicating with
  example:

```
https://api.com
```

Endpoint :

The specific route that gives me a certain Data

- the **Endpoint** is the extra path added after the base url to tell the backend what exactly i want

```
https://api.com/users/profile
```

now finalyyyyyyy the backend understands what i want
wow... communication skills... ;)

Headers :

Extra information attached with the request

- headers are some additional information that i send with the request to help the backend understand things like:
  - who am i?
  - what type of data am i sending?

```
{   "Content-Type": "application/json",   "Authorization": "Bearer TOKEN"}
```

Body :

The actual data that i want to send

- the **Body** usually exists in requests like **POST** , **PATCH** , and **PUT**
- it contains the actual data that i want to send to the backend

```
{   "name": "Basmala",   "email": "basmala@gmail.com"}
```

Query Parameters :

Small filters attached to the URL

- query parameters are extra values added to the url to filter or customize the returned data

```
https://api.com/songs?genre=rock&page=2
```

- `genre=rock` → only return rock songs
- `page=2` → return the second page

Call Flow :

The journey of the request itself

- the **Call Flow** means :
  how the request moves from the UI until it reaches the backend and then returns back again and then the response travels ALL the way back again until the UI finally displays the data

---

Data Flow :

How the actual data moves inside the app

- the **Data Flow** focuses on the data itself and how it travels through the layers of the application

---

Tokens :

The VIP card of authentication

- a **Token** is like a temporary identity card that proves to the backend that the user already logged in successfully
  that is not how security works unfortunately
